K.R. Mangalam University

School of Engineering & Technology

Operating System Lab

(ENCA252)

Lab Assignment 4

Scheduling Algorithms

Submitted by

Name: Akash Sharma

Roll No: 2401201108

Course: BCA (Ai & Ds)

FCFS (First-Come, First-Served)

How it works:

Processes requests in the exact order they arrive in the disk queue.

Logic:

Simple and fair, but doesn't optimize head movement at all. Each request is served regardless of how far the disk head must travel, which can lead to wild swings across the disk surface and high cumulative seek distances.

> ⚠ **FCFS is the simplest but worst-performing algorithm when requests are scattered across the disk.**

Step-by-Step Trace (Head: 53)

Step	From	To	Seek Distance
1	53	98	45
2	98	183	85
3	183	37	146
4	37	122	85
5	122	14	108
6	14	124	110
7	124	65	59
8	65	67	2

Total Seek Time: 640

Output:

```
akash@ASUS-TUF-Gaming-A15:~$ nano disk-scheduling.py
akash@ASUS-TUF-Gaming-A15:~$ python disk-scheduling.py
Enter request queue (space separated): 98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter disk size: 200

Input Summary:
Requests: [98, 183, 37, 122, 14, 124, 65, 67]
Initial Head: 53
Disk Size: 200

================================================
               FCFS Disk Scheduling
================================================
Step  | From   | To     | Seek Distance
------------------------------------------------
1     | 53     | 98     | 45
2     | 98     | 183    | 85
3     | 183    | 37     | 146
4     | 37     | 122    | 85
5     | 122    | 14     | 108
6     | 14     | 124    | 110
7     | 124    | 65     | 59
8     | 65     | 67     | 2

Total Seek Time: 640
```

SSTF (Shortest Seek Time First)

How it works:

Selects the request with the minimum seek distance from the current head position at each step.

Logic:

A greedy approach that always picks the nearest unvisited track. This dramatically reduces total seek time compared to FCFS, but can cause **starvation** for requests that are consistently far from the head whenever closer requests keep arriving.

> 🔥 **SSTF is best when disk requests are clustered. It can degrade to $O(n^2)$ in worst-case starvation scenarios.**

Step-by-Step Trace (Head: 53)

Step	From	To	Seek Distance
1	53	65	12
2	65	67	2
3	67	37	30
4	37	14	23
5	14	98	84
6	98	122	24
7	122	124	2
8	124	183	59

Total Seek Time: 236

Output:

```
================================================
               SSTF Disk Scheduling
================================================
Step  | From   | To     | Seek Distance
------------------------------------------------
1     | 53     | 65     | 12
2     | 65     | 67     | 2
3     | 67     | 37     | 30
4     | 37     | 14     | 23
5     | 14     | 98     | 84
6     | 98     | 122    | 24
7     | 122    | 124    | 2
8     | 124    | 183    | 59

Total Seek Time: 236
```

SCAN (Elevator Algorithm)

How it works:

The head moves in one direction (upward), servicing all requests along the way. Once it reaches the end of the disk (track 199), it reverses direction and services remaining requests on the way back.

Logic:

Provides more uniform wait times than SSTF and completely avoids starvation. Every request is guaranteed to be served within two full sweeps of the disk, making it suitable for high-load systems.

> **Often called the "Elevator Algorithm" because it behaves exactly like an elevator — going up, stopping at each floor with a request, then reversing.**

Step-by-Step Trace (Head: 53, Direction: Up)

Step	From	To	Seek Distance
1	53	65	12
2	65	67	2
3	67	98	31
4	98	122	24
5	122	124	2
6	124	183	59
7	183	199	16
8	199	37	162
9	37	14	23

Total Seek Time: 331

Output:

```
================================================
            SCAN Disk Scheduling (UP)
================================================
Step  | From   | To     | Seek Distance
------------------------------------------------
1     | 53     | 65     | 12
2     | 65     | 67     | 2
3     | 67     | 98     | 31
4     | 98     | 122    | 24
5     | 122    | 124    | 2
6     | 124    | 183    | 59
7     | 183    | 199    | 16
8     | 199    | 37     | 162
9     | 37     | 14     | 23

Total Seek Time: 331
```

C-SCAN (Circular SCAN)

How it works:

Similar to SCAN, but when the head reaches the end of the disk, it immediately jumps back to track 0 without servicing any requests on the return trip, then resumes servicing in the upward direction.

Logic:

Treats the disk tracks as a circular list, which provides even more uniform waiting times than SCAN. Requests near the beginning of the disk are never starved by the long reverse sweep.
The jump from end to start counts as head movement.

> C-SCAN is preferred in real-world OS implementations (e.g., Linux's CFQ scheduler) because it guarantees a bounded worst-case wait time.

Step-by-Step Trace (Head: 53, Direction: Up)

Step	From	To	Seek Distance
1	53	65	12
2	65	67	2
3	67	98	31
4	98	122	24
5	122	124	2
6	124	183	59
7	183	199	16
8	199	0	199
9	0	14	14
10	14	37	23

Total Seek Time: 382

Output:

```
================================================
              C-SCAN Disk Scheduling
================================================
Step  | From   | To     | Seek Distance
------------------------------------------------
1     | 53     | 65     | 12
2     | 65     | 67     | 2
3     | 67     | 98     | 31
4     | 98     | 122    | 24
5     | 122    | 124    | 2
6     | 124    | 183    | 59
7     | 183    | 199    | 16
8     | 199    | 0      | 199
9     | 0      | 14     | 14
10    | 14     | 37     | 23

Total Seek Time: 382
```

Performance Comparison

Algorithm	Total Seek Time	Efficiency Rank
SSTF	236	1st (Best)
SCAN	331	2nd
C-SCAN	382	3rd
FCFS	640	4th (Worst)

Output:

```
==================================================
ALGORITHM            | TOTAL SEEK TIME
--------------------------------------------------
FCFS                 | 640
SSTF                 | 236
SCAN                 | 331
C-SCAN               | 382
==================================================

Best Algorithm  : SSTF
Worst Algorithm : FCFS
```

☑ Conclusion

- **Best Performer: SSTF** achieved the lowest total seek time (236) for this request set. It is highly efficient when requests are localized.

- **Worst Performer: FCFS** was the least efficient (640), blindly following arrival order with no regard for head position, causing excessive back-and-forth movement.

- **Observations:** While SSTF is the fastest numerically, **SCAN** and **C-SCAN** are preferred in production systems due to better fairness and guaranteed starvation prevention. C-SCAN's circular approach offers the most uniform waiting time across all disk tracks.